LOG LENS: AN AI-POWERED BROWSER EXTENSION FOR INTELLIGENT LOG ANALYSIS AND ROOT-CAUSE EXPLANATION

A PROJECT REPORT PHASE I

Submitted by

ANISH RAJ P (312323247011)

ISRAEL PAUL K (312323247048)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

St. JOSEPH'S COLLEGE OF ENGINEERING

(An Autonomous Institution)

ANNA UNIVERSITY :: CHENNAI 600025

NOVEMBER 2026

ANNA UNIVERSITY : CHENNAI 600 025



BONAFIDE CERTIFICATE

Certified that this project report for **"LOG LENS: AN AI-POWERED BROWSER EXTENSION FOR INTELLIGENT LOG ANALYSIS AND ROOT-CAUSE EXPLANATION"** is the bonafide work of **ANISH RAJ P (312323247011)** and **ISRAEL PAUL K (312323247048)** who carried out the project under my supervision.

SIGNATURE

Dr. Adlin Sheeba, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Artificial Intelligence and
Machine Learning,
St. Joseph's College of Engineering,
Old Mamallapuram Road,
Chennai – 600 119.

SIGNATURE

Ms. J.A. Adlin Layola, M.E.,(Ph.D).,

SUPERVISOR,
ASSISTANT PROFESSOR,

Artificial Intelligence and
Machine Learning,
St. Joseph's College of Engineering,
Old Mamallapuram Road,
Chennai – 600 119.

ACKNOWLEDGEMENT

We also take this opportunity to thank our respected and honourable Chairman **Dr. B. Babu Manoharan M.A., M.B.A., Ph.D.,** for the guidance he offered during our tenure in this institution.

We extend our heartfelt gratitude to our respected and honourable Managing Director **Mr. B. Sashi Sekar M.Sc.,** for providing us with the required resources to carry out this project.

We express our deep gratitude to our honourable Executive Director **Mrs. S. Jessie Priya M.Com.,** for the constant guidance and support for our project.

We are indebted to our Principal **Dr. Vaddi Seshagiri Rao M.E., M.B.A., Ph.D.,** for granting us permission to undertake this project.

We would like to express our earnest gratitude to our Head of the Department **Dr. Adlin Sheeba M.E., Ph.D.,** for her commendable support and encouragement for the completion of the project with perfection.

We also take the opportunity to express our profound gratitude to our guide **Ms. J.A. Adlin Layola, M.E.,(Ph.D).,** for her guidance, constant encouragement, immense help and valuable advice for the completion of this project.

We wish to convey our sincere thanks to all the teaching and non-teaching staff of the department of **ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING** without whose cooperation this venture would not have been a success.

CERTIFICATE OF EVALUATION

College Name: ST. JOSEPH'S COLLEGE OF ENGINEERING
Branch: ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
Semester: VII

Sl.No	Name of the Students	Title of the Project	Name of the Supervisor with designation
1 2	ANISH RAJ P (312323247011) ISRAEL PAUL K (312323247048)	LOG LENS: AN AI-POWERED BROWSER EXTENSION FOR INTELLIGENT LOG ANALYSIS AND ROOT-CAUSE EXPLANATION	Ms. J.A. Adlin Layola, M.E.,(Ph.D)., Assistant Professor

The report of the project work submitted by the above students in partial fulfillment for the award of Bachelor of Technology Degree in Artificial Intelligence and Machine Learning of Anna University were evaluated and confirmed to be a report of the work done by above students.

Submitted for project review and viva voce exam held on _____

(INTERNAL EXAMINER) **(EXTERNAL EXAMINER)**

ABSTRACT

Modern software systems generate large volumes of log data that are difficult to interpret without explicit source-code context, inflating Mean Time to Resolution (MTTR). Conventional tools focus on aggregation and keyword search, offering limited support for semantic correlation and line-level root-cause explanations. To address these challenges, this project presents Log Lens, an intelligent log analysis system implemented as a browser extension for developer-centric debugging. Log Lens projects log entries and source code into a shared vector space via dense sentence-transformer embeddings and stores the code embeddings in a ChromaDB vector database. A hybrid temporal re-ranking algorithm then combines semantic similarity, Git commit recency, and stack frame depth scoring to prioritize probable regression locations. The top-ranked code context is assembled into a structured prompt and passed to a Large Language Model, which generates precise root-cause explanations with exact file and line references. The architecture consists of three tiers: a Chrome Extension Manifest V3 frontend that captures logs, a Node.js middleware layer that fetches Git blame information and surrounding code chunks from GitHub, and a Python FastAPI backend that orchestrates vector search, hybrid re-ranking, and LLM prompting.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	**iii**
	CERTIFICATE OF EVALUATION	**iv**
	ABSTRACT	**v**
	LIST OF FIGURES	**ix**
	LIST OF ABBREVIATIONS	**x**
1	**INTRODUCTION**	**1**
	1.1 SYSTEM OVERVIEW	1
	1.2 MOTIVATION	3
	1.3 PROBLEM STATEMENT	4
	1.4 SCOPE OF THE PROJECT	5
2	**LITERATURE SURVEY**	**6**
3	**SYSTEM ANALYSIS**	**10**
	3.1 EXISTING SYSTEM	10
	3.1.1 Disadvantages of Existing System	11
	3.2 PROPOSED SYSTEM: LOG LENS	12
	3.2.1 Log Ingestion Pipeline	13
	3.2.2 Semantic Retrieval Module	14
	3.2.3 Temporal Re-Ranking Module	14

3.2.4 Log-to-Code Mapping 15

3.2.5 LLM Explanation Generation 16

3.2.6 Browser Extension Interface 16

3.2.7 Advantages of the Proposed System 17

3.3 REQUIREMENTS SPECIFICATION 18

3.3.1 Hardware Requirements 18

3.3.2 Software Requirements 19

4 SYSTEM ARCHITECTURE 21

4.1 SYSTEM FLOW 21

4.1.1 Overall System Architecture 21

4.1.2 Log Ingestion and Chunking Pipeline 22

4.1.3 Semantic Retrieval and Re-Ranking Flow 24

4.1.4 LLM Explanation Generation Flow 26

5 SYSTEM IMPLEMENTATION 29

5.1 LOG NORMALIZATION AND MULTI-SIGNAL HEURISTICS 29

5.2 SLIDING WINDOW CHUNKING STRATEGY 29

5.3 EMBEDDING GENERATION USING ALL-MINILM-L6-V2 30

5.4 HYBRID RE-RANKING SCORING ALGORITHM 30

6 CONCLUSION AND FUTURE WORK 31

6.1 CONCLUSION 31

6.2 FUTURE WORK 32

APPENDIX 33

REFERENCES 37

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Overall System Architecture of Log Lens	22
4.2	Log Ingestion and Chunking Pipeline	24
4.3	Semantic Retrieval and Temporal Re-Ranking Flow	26
4.4	LLM Explanation Generation and Response Rendering Flow	28

LIST OF ABBREVIATIONS

ABBREVIATION	DEFINITION
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Deployment
ChromaDB	Chroma Vector Database
DOM	Document Object Model
FastAPI	Fast Application Programming Interface (Python async framework)
GCP	Google Cloud Platform
Git	Distributed Version Control System
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
OAuth	Open Authorization
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 SYSTEM OVERVIEW

Log Lens is a browser extension that embeds AI-powered log diagnosis directly into the developer's existing workflow by capturing log output from cloud consoles and Docker environments, connecting to the user's GitHub repository, and automatically identifying the root cause of errors at the exact file and line level. Rather than presenting raw diagnostic data, Log Lens generates a plain-English explanation of what went wrong, why it happened, and where in the codebase the fault originates. The system is designed to be transparent and non-intrusive: it requires no modifications to application code, no changes to deployment pipelines, and no dedicated infrastructure beyond the extension itself and a lightweight backend server.

Log Lens is built on a three-tier architecture. The first tier is a browser extension implemented using the Chrome Extension Manifest V3 API. This extension injects a content script into cloud console pages (such as the AWS CloudWatch Logs console and the GCP Cloud Logging console) and provides a proxy integration for Docker terminal output. The content script monitors the page DOM for log output elements and captures log entries as they appear. When an error-level signal is detected — identified by the presence of stack trace patterns, ERROR-level log entries, or HTTP 5xx status codes — the extension captures a window of surrounding log context and transmits it to the Node.js middleware via a local REST call.

The second tier is a Node.js middleware server that acts as the orchestration layer between the extension and the analysis backend. Upon receiving a log bundle from the extension, the middleware authenticates with the GitHub API

using the user's stored OAuth token, resolves the repository associated with the current project, fetches the Git blame information for the files referenced in the stack trace, and retrieves the surrounding source code chunks for those files. The middleware bundles the raw log data, the parsed stack trace, the fetched code chunks, and the Git metadata into a structured JSON payload and forwards it to the FastAPI backend for analysis.

The third tier is a Python FastAPI backend that implements the core AI analysis pipeline. The backend receives the structured bundle from the middleware and passes the error signal through an embedding model to generate a dense vector representation. This vector is used to query a ChromaDB vector database that has been pre-populated with embeddings of all code chunks from the connected GitHub repository. The top-K semantically similar code chunks are retrieved and passed to a temporal re-ranking module that adjusts their scores using Git metadata signals. The final ranked context is assembled into a structured prompt and submitted to a Large Language Model, which generates a root-cause explanation identifying the responsible file and line number. The explanation is returned as a JSON object to the middleware, which forwards it to the extension for rendering in the sidebar panel.

The main capabilities of Log Lens are:

- Automated root-cause identification from raw log output without any manual analysis by the developer.
- Precise line-level attribution that links errors to specific source files and line numbers in the GitHub repository.
- Human-readable natural language explanations accessible to developers of all experience levels.
- Temporal re-ranking that prioritises recently modified code, reducing false attribution to stable library code that is statistically unlikely to be the source

of a new defect.

- Seamless integration with existing cloud platforms and GitHub repositories through OAuth-based authentication, requiring no changes to application code or deployment infrastructure.

1.2 MOTIVATION

The widespread adoption of cloud-native architectures and containerized deployment pipelines has fundamentally changed how modern software is built, operated, and maintained. Platforms such as Amazon Web Services, Google Cloud Platform, and Docker generate enormous volumes of log data as a natural byproduct of application execution. These logs record system behavior at every layer of the stack — from kernel events and network connections to application exceptions and API responses — and represent the primary diagnostic artifact available to engineers when a failure occurs.

Despite their importance, logs are extraordinarily difficult to use effectively in practice. A single Docker container running a Node.js web service can produce tens of thousands of log lines per hour under moderate traffic. When an exception is thrown, the relevant error signal is typically buried within this output, surrounded by informational messages from unrelated code paths, preceded by stack traces that reference internal library frames rather than application code, and mixed with log streams from other co-located services. The cognitive effort required to manually parse this output, identify the relevant signal, and trace it back to the specific line of source code that caused the failure is substantial even for experienced engineers, and can be prohibitive for less experienced developers. This gap between raw runtime log streams and actionable, code-grounded diagnostic explanations directly motivates the development of Log Lens.

1.3 PROBLEM STATEMENT

When a software deployment fails or a running application throws an exception, the developer is presented with a wall of log output. In production environments these logs may span thousands of lines, interleave messages from multiple services, and contain stack traces referencing internal library code rather than the application's own source. The developer must mentally parse this output, identify the relevant error signal among noise, map it to the correct file and line in the source repository, and reason about why that specific code failed given the current runtime context.

This manual debugging process has several well-documented problems. First, it is slow: even experienced engineers regularly spend 30 to 60 minutes diagnosing a single production error from logs alone. Second, it scales poorly with system complexity, since microservice and cloud-native architectures produce distributed traces that span multiple independent log streams with no single unified view. Third, it is error-prone: developers frequently misidentify the root cause because the visible exception is a downstream symptom of an upstream failure in a different service or code path. Fourth, it is inaccessible to junior developers who lack the contextual knowledge to interpret cryptic runtime exception messages or navigate unfamiliar codebases.

Existing commercial tools address the aggregation and search aspects of this problem. Datadog, Splunk, and AWS CloudWatch provide powerful capabilities for ingesting, indexing, and querying log data at scale, and can be configured to alert on specific log patterns. However, these tools are fundamentally search and display systems. They surface the raw log data efficiently but provide no interpretation of what the log entries mean in relation to the application's source code. The developer is left with exactly the same diagnostic task as before, just with a more navigable interface for finding the relevant entries. There is a clear and unaddressed need for a system that can close the gap between raw log output and a

human-understandable, code-referenced root-cause diagnosis.

1.4 SCOPE OF THE PROJECT

The scope of the Phase I implementation covers the complete pipeline from log capture to explanation rendering, evaluated on Docker and standard cloud log formats. The log ingestion module reads and normalises raw log output from Docker container logs and AWS CloudWatch log streams, parsing stack traces and chunking the log content into semantically coherent units. The semantic retrieval module embeds both log chunks and source code file chunks into a shared vector space using a sentence transformer model and uses ChromaDB for approximate nearest-neighbour search. The temporal re-ranking module augments semantic similarity scores with Git commit recency weights, stack trace depth scores, and author activity signals derived from the GitHub API. The LLM explanation module constructs a structured prompt from the ranked context and generates a JSON-structured root-cause explanation. The browser extension renders the explanation in a sidebar panel with a direct link to the identified file and line in GitHub.

Evaluation for Phase I is conducted on 20 to 30 real error cases collected from Docker environments and cloud-deployed applications, measuring explanation accuracy (whether the correct root cause is identified), line-pointer precision (whether the identified file and line are correct), and end-to-end response latency. The project is designed with a clear extension path for Phase II, which will expand the benchmark to 100 or more cases, add multi-cloud comparison across AWS, GCP, and Docker log formats, conduct a user study with 10 or more developers, perform ablation analysis comparing semantic-only retrieval against temporal-only retrieval against the full hybrid approach, and provide a latency and scalability analysis.

CHAPTER 2

LITERATURE SURVEY

Cheng et al. [3] (2026) presented a survey of log anomaly detection methods based on Large Language Models. The survey organises how LLMs are used to represent and analyse log data and discusses the strengths and open problems of these methods. It confirms the shift from purely statistical and rule-based processing towards semantically aware log analysis, which motivates the embedding-based log representation used in Log Lens. As a survey, it does not describe a mechanism that links log events to the responsible source code.

Tran et al. [13] (2026) presented an LLM-augmented knowledge base construction approach for root-cause analysis. By building a retrievable knowledge base that grounds the LLM's diagnosis, their work supports the retrieval-augmented principle at the heart of Log Lens, where code chunks stored in a vector database serve as the external knowledge that grounds each explanation. Unlike Log Lens, their knowledge base is built from support tickets and not from the developer's own repository and its commit history.

Tan et al. [12] (2026) proposed LLMRCA, a multilevel root-cause analysis method for LLM applications that uses multimodal observability data. By combining several signal types, the approach shows that diagnosis improves when evidence from multiple sources is correlated, an idea reflected in the multi-signal re-ranking of Log Lens. Their target, however, is the operational monitoring of LLM applications rather than interactive per-developer debugging.

Sergeyuk et al. [11] (2026) presented a longitudinal analysis of developer logs to study how AI assistance changes everyday developer workflows. The

study underlines that development remains a context-heavy activity in which developers must relate observed behaviour to recent changes in the codebase. This observation motivates the temporal re-ranking mechanism of Log Lens, which uses Git commit recency as a proxy for defect likelihood and prioritises recently modified code.

Huang et al. [6] (2025) proposed LogRules, which improves the log analysis capability of LLMs by inducing rules from log data and injecting them into the model. Their work shows that guiding an LLM with explicit rules improves its handling of log parsing and anomaly detection. The limitation for the Log Lens use case is that the rules describe log semantics only and do not connect a detected error to repository files or lines.

Ma et al. [7] (2025) introduced LibreLog, an accurate and efficient unsupervised log parsing approach built on open-source LLMs. LibreLog demonstrates that open-source models can parse logs effectively without labelled data, which informs the Log Lens ingestion strategy and its privacy-oriented outlook. However, parsing produces structured templates and does not explain the root cause of an error.

Mollik et al. [9] (2025) introduced RCEGen, a generative approach to automated root-cause analysis using LLMs that produces natural-language explanations of faults. RCEGen is closely aligned with the explanation-generation goal of Log Lens. It does not, however, retrieve source code or use version control history to pinpoint the file and line responsible.

Mhatre and Shaikh [8] (2025) applied LLMs to automated root-cause analysis in microservices architectures, where failures propagate across independent services and a single visible exception may be a downstream symptom. This confirms the difficulty of attributing a fault from a single log stream and supports the Log Lens focus on ranking probable regression locations.

Tuli et al. [14] (2025) examined AI-assisted debugging tools for enhancing developer productivity in large codebases using genetic programming algorithms, indicating that tool support for locating and fixing faults becomes more valuable as codebases grow. This supports the Log Lens aim of reducing Time-to-Fix, though such tools generally remain separate from the live log stream that the developer is inspecting.

De la Cruz Cabello et al. [4] (2025) conducted a systematic literature review of AIOps for log anomaly detection in the era of LLMs. The review documents the growing role of LLMs in operations tooling, but anomaly detection alone leaves the developer to trace the underlying cause manually.

Aboudrar et al. [1] (2025) presented an AI-driven firewall log analysis approach that uses deep learning to enhance threat detection. Their work shows that learned models can extract useful signals from large log volumes, while its focus on detection rather than explanation leaves the code-level attribution problem unaddressed.

Xiao et al. [15] (2024) proposed a demonstration-free approach towards more practical log parsing with LLMs, reducing the dependence on labelled examples that limits many LLM-based parsers. Their findings reinforce the need for a lightweight client footprint, a goal followed by the Log Lens browser extension, although the work stops at parsing and does not localise faults in code.

Roy et al. [10] (2024) explored LLM-based agents for root-cause analysis, equipping a ReAct-style agent with retrieval and diagnostic tools so that it can gather information dynamically during an investigation. Their work highlights the promise of LLMs for root-cause analysis, while also motivating a structured retrieval stage that limits the context handed to the model.

Chatterjee and Das [2] (2024) investigated AI-powered anomaly detection for real-time performance monitoring in cloud systems, showing that intelligent

analysis can be applied continuously to live monitoring data. This supports the Log Lens approach of analysing logs from cloud consoles as they appear, through a browser extension that overlays diagnostic assistance on existing interfaces without requiring changes to the development environment or application code.

Hasanov et al. [5] (2024) conducted a systematic literature review of the application of LLMs in cybersecurity, documenting both the breadth of LLM applications and the concerns around their use, including data sensitivity. These concerns inform the Log Lens design decision to send only a bounded, ranked code context to the LLM and motivate the on-device inference planned for Phase II.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing landscape of developer log analysis tools can be broadly classified into three categories: log aggregation and search platforms, static analysis tools, and general-purpose AI coding assistants. Each category addresses part of the developer debugging problem but leaves the most critical step — translating raw log output into a code-referenced, human-understandable root-cause diagnosis — entirely to the developer.

Log aggregation platforms such as Datadog, Splunk, Elastic Stack (ELK), and AWS CloudWatch provide powerful capabilities for ingesting, indexing, and querying log data at scale. These tools allow developers to filter logs by time range, severity level, and keyword, and to configure alerting rules that fire when specific patterns are detected. AWS CloudWatch Logs Insights provides a query language for structured log analysis and can compute metrics and visualisations from log data. Despite these capabilities, these tools are fundamentally search and display systems. They surface the relevant log entries efficiently but provide no interpretation of what those entries mean in the context of the application's source code. The developer receives the same raw error message they would have seen in the terminal — just more conveniently searchable.

Static analysis tools such as SonarQube, Checkmarx, and ESLint analyse source code before deployment to identify potential bugs, security vulnerabilities, and code quality issues. These tools are valuable for preventing defects but operate entirely on source code and have no visibility into runtime log data. They cannot identify the cause of an error that

manifests only under specific runtime conditions such as a particular database state, a race condition, or an unexpected input value.

General-purpose AI coding assistants such as GitHub Copilot, Cursor, and direct use of ChatGPT or Claude can be queried about error messages and code snippets with increasing effectiveness. However, using these tools for log debugging requires the developer to manually extract the relevant log excerpt, locate the relevant source code in their IDE, and construct an appropriate query. There is no automatic connection between the live log stream and the source repository. The AI assistant has no knowledge of the project's Git history, no context about which files were recently changed, and no ability to perform semantic retrieval over the entire codebase to find the most relevant code for a given error.

3.1.1 Disadvantages of Existing System

- No Automatic Log-to-Code Linkage: Existing tools do not automatically connect log error traces to the specific lines of source code responsible for the failure. The developer must manually navigate from log output to the correct file and line, a process that requires deep familiarity with the codebase.
- No Root-Cause Explanation: Log aggregation platforms display what failed but not why it failed. A stack trace showing a NullPointerException at a specific line tells the developer where the failure was detected but not what upstream condition caused the null value to be present at that point.
- No Integration of Version Control Context: No existing mainstream tool incorporates Git commit history as a signal for fault localization. The empirically established insight that recently modified files are significantly more likely to be the source of a new error is not reflected in any current log analysis tool.

- Context Switching Overhead: Using AI assistants for log debugging requires the developer to leave the cloud console, open a chat interface, manually copy the relevant log excerpt and code snippet, and formulate a query. This context switching disrupts the debugging workflow and increases time to resolution.

- Inability to Handle Log Volume: In production environments, relevant error signals are submerged in thousands of informational log lines from unrelated code paths. Keyword search is insufficient when the causal connection between the error and its source is semantic rather than lexical.

- No Temporal Awareness: When a new deployment fails, the most likely cause is code that was recently changed. Existing tools return results ranked by textual similarity or time, not by the combination of semantic relevance and recent change activity that would prioritise the most defect-likely code.

- High Cost and Operational Overhead: Enterprise log platforms such as Splunk and Datadog require dedicated infrastructure, ongoing administration, and significant licensing cost. Individual developers and small teams often cannot justify this overhead for their own project debugging needs.

- No Multi-Platform Unification: Docker logs, AWS CloudWatch, and GCP Cloud Logging each use different formats and require separate tooling and configuration. There is no unified interface that handles all three with a consistent analysis pipeline.

3.2 PROPOSED SYSTEM: LOG LENS

Log Lens is an AI-powered browser extension that addresses each of the limitations described above through a unified pipeline that combines log ingestion and normalisation, semantic embedding-based retrieval using ChromaDB, temporal re-ranking using Git metadata, log-to-code mapping

at the file and line level, and LLM-based natural language explanation generation. The system is designed to operate as a zero-configuration overlay on the developer's existing cloud console workflow.

The core technical innovation of Log Lens is the combination of two complementary ranking signals: semantic similarity between the error representation and source code chunks, and temporal recency of code changes from Git history. Semantic retrieval alone can connect an error message to conceptually related code anywhere in the repository but may rank stable library files above recently modified application files. Temporal re-ranking corrects this bias by boosting scores for files modified in recent commits, particularly those directly referenced in the stack trace. This hybrid approach is designed to achieve higher root-cause attribution accuracy than either signal alone, and is one of the key contributions to be validated in the evaluation.

3.2.1 Log Ingestion Pipeline

The log ingestion pipeline begins when the browser extension content script detects an error signal in the monitored log output. The detection heuristic identifies error-level entries using a combination of severity label matching (ERROR, FATAL, CRITICAL, Exception, Traceback), HTTP status code detection for 5xx responses, and stack trace pattern recognition for common runtime exception formats in Java, Python, Node.js, and Go.

When an error is detected, the extension captures a configurable context window of log lines surrounding the error (default: 50 lines before and after the triggering entry) and transmits the captured window to the Node.js middleware. The middleware performs log normalisation, converting Docker JSON log format, AWS CloudWatch log event format, and GCP structured log entry format into a unified internal representation containing a timestamp, severity level, raw message text, and any parsed structured fields. Stack traces

are extracted as a distinct structured field and parsed to identify referenced file names, class names, method names, and line numbers where present.

The normalised log is chunked using a sliding window strategy with a default chunk size of 20 lines and a 5-line overlap between adjacent chunks. Stack traces are preserved as atomic chunks to prevent a single trace from being split across two retrieval units and losing the structural integrity of the call sequence. The chunks, along with the parsed stack trace fields, are forwarded to the FastAPI backend in the structured bundle.

3.2.2 Semantic Retrieval Module

The semantic retrieval module receives the log bundle from the middleware and generates a dense vector embedding of the primary error chunk using the all-MiniLM-L6-v2 sentence transformer model, which produces a 384-dimensional vector. This model was selected for its strong performance on semantic similarity benchmarks combined with low inference latency (approximately 5ms per chunk on CPU), which is necessary for the interactive use case.

Source code files from the connected GitHub repository are pre-indexed offline. Each file is split into overlapping chunks of 50 lines with a 10-line overlap, embedded using the same model, and stored in ChromaDB. When an error query arrives, the embedded error vector is submitted to ChromaDB for approximate nearest-neighbour search, returning the top-20 semantically similar code chunks with their cosine similarity scores. These 20 candidates are passed to the temporal re-ranking module.

3.2.3 Temporal Re-Ranking Module

The temporal re-ranking module computes a composite score for each of the 20 candidate code chunks by combining the semantic similarity score with signals derived from the repository's Git history. For each candidate file, the

module queries the GitHub API to obtain the timestamp of the most recent commit to the file, the number of commits to the file in the past 30 days, and the Git blame mapping for the lines within the candidate chunk.

A stack depth score is assigned to each candidate based on how prominently its file appears in the parsed stack trace: files at the top of the stack trace (nearest to the point of failure) receive the highest stack depth score, files deeper in the call stack receive lower scores, and files not appearing in the stack trace receive a score of zero. The composite re-ranking score is computed as a weighted linear combination: 0.4 multiplied by the semantic similarity score, plus 0.35 multiplied by the recency weight (computed as 1 divided by 1 plus the number of days since the last commit), plus 0.25 multiplied by the stack depth score. The top-5 candidates after re-ranking are selected as the context for LLM explanation.

3.2.4 Log-to-Code Mapping

The log-to-code mapping module performs precise linkage between specific entries in the log output and specific locations in the source repository. Stack trace lines are parsed using language-specific regular expressions to extract file names, class names, method names, and line numbers. These extracted references are resolved against the repository's file tree using the GitHub API to obtain the exact source file at the commit currently deployed.

For errors that do not include explicit stack traces — such as application-level error log messages that describe a semantic failure without a call stack — the semantic retrieval results provide a probabilistic mapping. The top-ranked code chunk from the re-ranking stage is taken as the most likely location of the root cause, and the LLM is instructed to identify the specific line within that chunk during explanation generation.

3.2.5 LLM Explanation Generation

The LLM explanation module receives the structured bundle containing the raw log excerpt, the parsed stack trace, the top-5 re-ranked code chunks with their file paths and line ranges, and the Git metadata for each candidate. A structured prompt is constructed with four sections: a system instruction defining the model's role as a software debugging assistant with access to the application's source code and recent log output, an error context section containing the raw log excerpt and parsed stack trace, a code context section presenting each of the five code chunks with its file path and line range clearly labelled, and an output format specification instructing the model to respond with a JSON object containing the fields root_cause_file, root_cause_line, error_summary (one sentence), and detailed_explanation (multi-sentence causal chain).

The JSON response from the LLM is parsed and validated. The root_cause_file value is checked against the repository's file tree to confirm it corresponds to a real file. The root_cause_line value is checked to confirm it falls within the line range of one of the provided code chunks. If validation fails, the module falls back to returning the top-ranked file and its line range without a specific line number. The validated response is returned through the middleware to the browser extension.

3.2.6 Browser Extension Interface

The browser extension provides a sidebar panel that activates automatically when an error is detected in the captured log stream. The panel displays four elements: the original error excerpt from the log highlighted for readability, the one-sentence error summary from the LLM, the detailed multi-sentence root-cause explanation, and a clickable link to the identified file and line number in the GitHub repository that opens the file at the exact location in a new tab. A confidence indicator displays the cosine similarity score of the

top-ranked code chunk, giving the developer a quick signal of how confident the retrieval result is.

The extension uses OAuth 2.0 with the PKCE flow to obtain a GitHub access token with read-only access to the user's repositories. Cloud platform access (AWS CloudWatch, GCP Cloud Logging) is obtained through each platform's respective OAuth flow. No credentials are stored on any external server; tokens are stored exclusively in the browser's local storage using the chrome.storage.local API.

3.2.7 Advantages of the Proposed System

- Automated Root-Cause Attribution: Log Lens automatically links log errors to the specific file and line responsible, eliminating the manual debugging step of navigating from log output to source code.
- Hybrid Temporal and Semantic Retrieval: The combination of semantic similarity and Git recency weighting prioritises recently modified code, reducing false attribution to stable library files and improving attribution accuracy over semantic search alone.
- Human-Readable Explanations: LLM-generated explanations describe the error and its causal chain in plain English, accessible to developers at all experience levels.
- Zero-Configuration Workflow Integration: As a browser extension, Log Lens overlays the developer's existing cloud console without requiring any changes to application code, deployment pipelines, or infrastructure.
- Multi-Format Log Support: The log normalisation pipeline handles Docker, AWS CloudWatch, and GCP Cloud Logging formats through a unified representation.
- Privacy-Preserving Authentication: OAuth tokens are stored only in the

browser's local storage and are never transmitted to or stored on any Log Lens server.

- Low Cost Deployment: The system requires only a Python FastAPI server and a Node.js middleware server, both of which can be run on a standard developer laptop or a small cloud VM at negligible cost.

3.3 REQUIREMENTS SPECIFICATION

A thorough understanding of the system's operational constraints is essential for ensuring that Log Lens functions correctly across the range of developer environments it targets. The requirements below reflect the practical constraints of running a three-tier AI analysis pipeline from a browser extension.

3.3.1 Hardware Requirements

Minimum Configuration:

- Processor: Intel Core i5 (8th generation) or higher
- RAM: 8 GB minimum system RAM
- Storage: 20 GB free SSD space for model weights (approximately 90 MB for all-MiniLM-L6-v2), the ChromaDB index, and Node modules
- Network: Stable internet connection for GitHub API calls, cloud log streaming, and LLM API access (if using a hosted LLM)
- Browser: Google Chrome version 100 or higher with Manifest V3 extension support

Recommended Configuration:

- Processor: Multi-core processor for parallel embedding and API handling
- RAM: 16 GB when running the LLM backend locally
- GPU: Optional NVIDIA GPU with CUDA 11.0+ support for accelerated sentence-transformer inference; CPU inference is also supported

3.3.2 Software Requirements

Runtime Environment:

- Python 3.10 or higher: FastAPI backend server and analysis pipeline
- Node.js 18.x LTS or higher: Middleware server for GitHub API, log routing, and bundle assembly
- Docker 20.10 or higher: Container runtime for testing Docker log ingestion and local backend deployment

Backend Frameworks and Libraries:

- FastAPI 0.100+: Async REST API framework for the /analyze endpoint
- ChromaDB 0.4+: Vector database for storing and querying code chunk embeddings
- sentence-transformers 2.2+: Semantic embedding of log chunks and code chunks (all-MiniLM-L6-v2)
- LangChain 0.2+: LLM prompt management and chain orchestration

GitHub Integration:

- Octokit (Node.js), latest: GitHub REST API client for file content, Git blame, and commit history

- PyGitHub, latest: Python GitHub API client for repository indexing

Browser Extension and LLM:

- Chrome Extension API (Manifest V3): Browser extension runtime, content scripts, and local storage
- LLM Backend (Claude / GPT-4): Root-cause explanation generation from ranked code context

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 SYSTEM FLOW

4.1.1 Overall System Architecture

The Log Lens system is organised into four functional tiers that execute sequentially for every detected error event. These tiers are: (1) log capture and error detection in the browser extension, (2) middleware orchestration that fetches Git and code context from GitHub, (3) backend AI analysis that performs embedding, vector retrieval, re-ranking, and LLM explanation generation, and (4) explanation rendering in the extension sidebar. The architecture is designed so that each tier has a single clear responsibility and communicates with adjacent tiers through well-defined REST API interfaces, making the system modular, testable, and extensible.

The data flow through the complete system, as specified in the project design, is as follows. The browser extension content script monitors cloud console DOM elements and Docker terminal output for error signals. When an error is detected, the extension captures a window of surrounding log context and POSTs it to the Node.js middleware. The middleware authenticates with the GitHub API using the stored OAuth token, resolves the repository, fetches Git blame information and surrounding code chunks for files referenced in the stack trace, and assembles a structured bundle containing the raw log, the parsed stack trace, an array of code chunks, and the associated Git metadata. This bundle is POSTed to the FastAPI /analyze endpoint. The Python backend embeds the error signal, queries ChromaDB for semantically similar code chunks, applies the hybrid temporal re-ranking algorithm, constructs the LLM prompt from the top-5 ranked candidates, and calls the LLM to

generate the root-cause explanation with a specific file and line pointer. The explanation is returned as a JSON object to the middleware, which forwards it to the extension, where it is rendered in the sidebar panel with a direct link to the identified location in GitHub.

The overall system architecture of Log Lens is illustrated in Figure 4.1, which shows the four tiers of the system and the data flow between them.

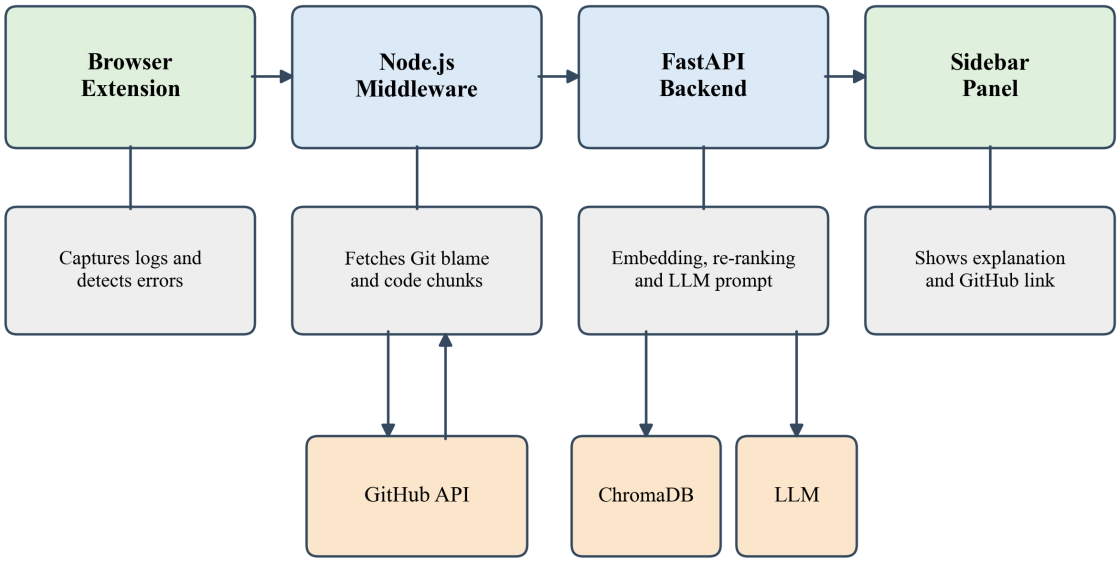


Figure 4.1: Overall System Architecture of Log Lens

4.1.2 Log Ingestion and Chunking Pipeline

The log ingestion pipeline is the entry point of the Log Lens system. The browser extension content script is injected into cloud console pages and monitors the page DOM for log output elements using a MutationObserver attached to the log display container. For Docker environments, a companion proxy script captures terminal standard output and standard error streams and forwards them to the extension via a local WebSocket connection on port 9000.

Error detection is performed using a multi-signal heuristic. The primary

signal is the presence of severity labels: the strings ERROR, FATAL, CRITICAL, Exception, Traceback, panic, and their common variants in multiple runtime environments (Java, Python, Node.js, Go) are matched using case-insensitive regular expressions. The secondary signal is HTTP status code detection: 5xx status codes appearing in API gateway or application server log entries trigger error capture. The tertiary signal is stack trace pattern recognition: the presence of consecutive lines matching the indentation and source reference patterns common to Python tracebacks, Java stack traces, Node.js exception output, and Go panic output is treated as an error signal independent of severity labels.

When an error is detected, the extension captures the immediately preceding 50 lines and the immediately following 20 lines as context, for a total context window of approximately 70 lines. This window is transmitted to the Node.js middleware as a POST request to the /log endpoint with a JSON body containing the raw text, the detected platform type, and the timestamp.

The middleware performs log normalisation by detecting the platform type and applying the appropriate parser. Docker JSON logs — which wrap each log line in a JSON object with fields for log text, stream, and time — are unpacked to extract the raw text and timestamp. AWS CloudWatch log events — returned by the GetLogEvents API as objects with message and timestamp fields — are extracted similarly. GCP structured log entries — which use the jsonPayload or textPayload structure — are parsed to extract the text message and severity. The normalised entries are assembled into a unified internal representation.

The normalised log window is chunked using a sliding window with a chunk size of 20 lines and a 5-line overlap. Stack trace blocks are identified by detecting the characteristic indentation pattern of exception stack frames and are preserved as single atomic chunks regardless of length. The chunked log data and the separately parsed stack trace fields are included in the bundle sent

to the FastAPI backend. The complete log ingestion and chunking pipeline is shown in Figure 4.2.

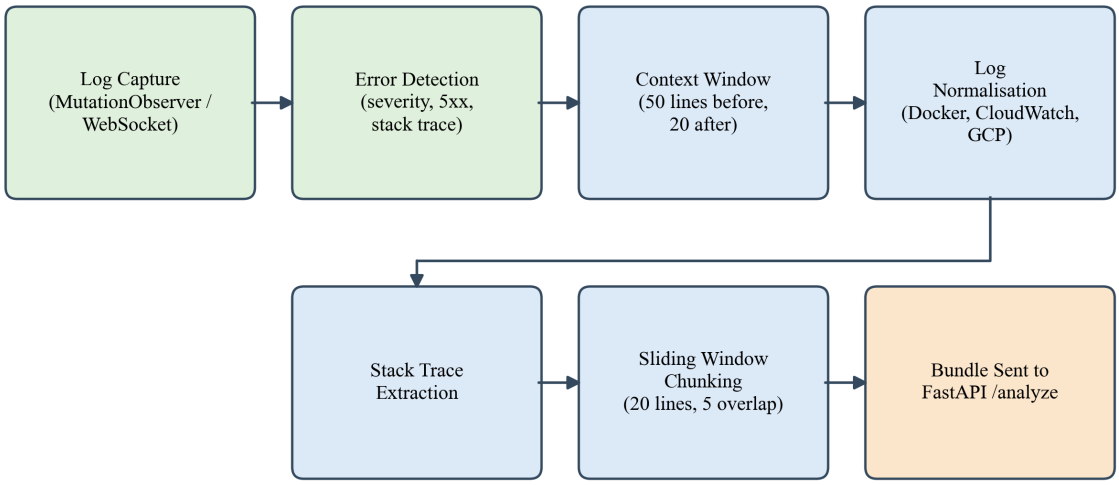


Figure 4.2: Log Ingestion and Chunking Pipeline

4.1.3 Semantic Retrieval and Re-Ranking Flow

The semantic retrieval and re-ranking pipeline is the core AI component of Log Lens. Upon receiving the structured bundle from the middleware at the /analyze endpoint, the FastAPI backend first extracts the primary error chunk — the chunk containing the highest-severity log entry or the most complete stack trace — and passes it through the all-MiniLM-L6-v2 sentence transformer model to generate a 384-dimensional dense vector embedding. The model is loaded at server startup and kept in memory to avoid repeated loading latency; inference time for a single 20-line chunk is approximately 5 to 10 milliseconds on CPU.

The generated query vector is submitted to ChromaDB for approximate nearest-neighbour search. The ChromaDB collection has been pre-populated with embeddings of all code chunks from the connected GitHub repository, computed offline using the same model. The collection uses the HNSW index type for efficient approximate search. The query returns the top-20

most similar code chunks ranked by cosine similarity, along with their stored metadata including file path, start line, end line, and the commit hash at which they were indexed.

The 20 retrieved candidates are then passed to the temporal re-ranking module. For each candidate, the module computes three component scores. First, the semantic similarity score is the cosine similarity value returned by ChromaDB, ranging from 0 to 1. Second, the recency weight is computed as 1 divided by (1 plus the number of days since the most recent commit to the candidate file), queried from the GitHub Commits API; this produces a weight approaching 1.0 for files modified today and decaying toward zero for files not modified in 30 or more days. Third, the stack depth score is computed by checking whether the candidate file appears in the parsed stack trace: files at the top frame of the stack trace receive a score of 1.0, files at subsequent frames receive a score that decreases with depth, and files not in the stack trace receive 0.0.

The composite re-ranking score is computed as a weighted sum: 0.4 multiplied by the semantic similarity score, plus 0.35 multiplied by the recency weight, plus 0.25 multiplied by the stack depth score. The 20 candidates are sorted by this composite score in descending order, and the top-5 are selected as the final context for LLM prompting. These 5 candidates, along with their file paths, line ranges, and Git metadata, are passed to the explanation generation module. The semantic retrieval and temporal re-ranking flow is illustrated in Figure 4.3.

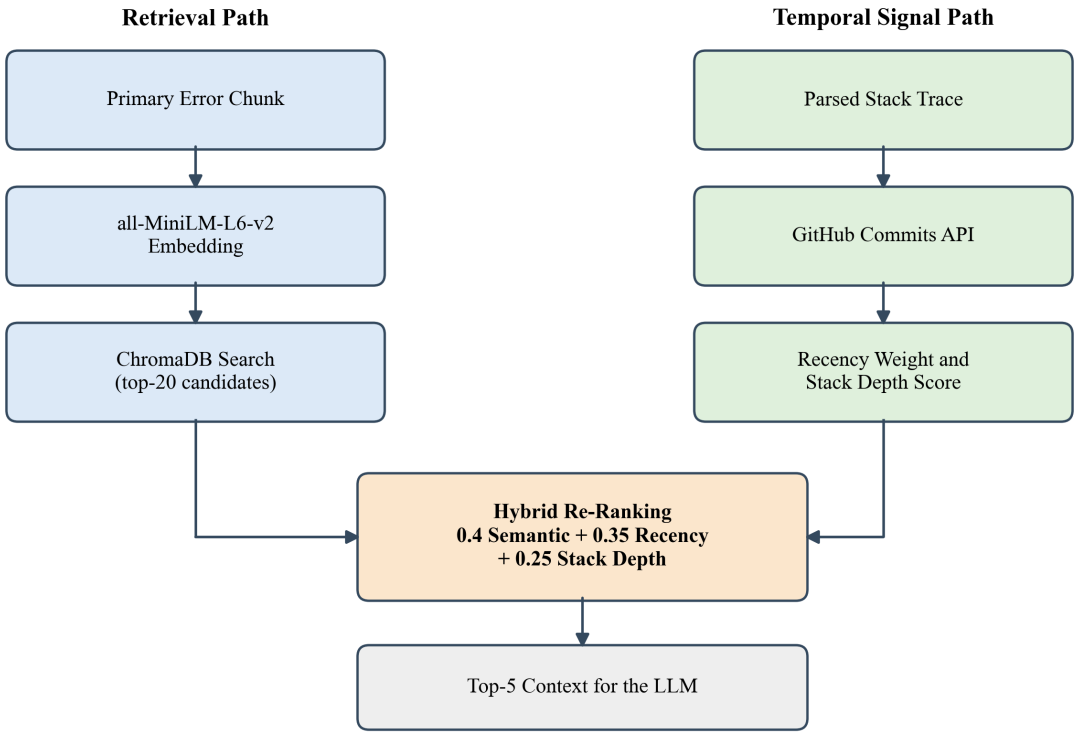


Figure 4.3: Semantic Retrieval and Temporal Re-Ranking Flow

4.1.4 LLM Explanation Generation Flow

The explanation generation module receives the top-5 re-ranked code chunks and constructs a structured prompt for submission to the LLM. The prompt is organised into four clearly delimited sections. The first section is the system instruction, which defines the model's role as an expert software debugging assistant that has access to log output from a running application and relevant source code from the associated repository. The instruction specifies that the model must identify the single most likely root cause, specify the exact file path and line number, and produce a structured JSON response.

The second section is the error context, which presents the raw log excerpt with line numbers preserved, followed by the structured stack trace fields if they were successfully parsed. The third section is the code context, which presents each of the five code chunks in sequence, each preceded by a header line identifying its file path, line range, cosine similarity score, recency

weight, and whether it appears in the stack trace. This structured presentation is designed to give the model the information it needs to assess which chunk is most likely to contain the root cause while preserving the raw code for inspection.

The fourth section is the output specification, which instructs the model to respond with a JSON object and no other text, with the following fields: root_cause_file (the full file path of the file containing the root cause), root_cause_line (the specific line number), error_summary (a single sentence describing what failed), and detailed_explanation (two to four sentences describing the causal chain from the application state to the observed error, written in plain English for a developer who may not be familiar with this specific module).

The LLM response is parsed as JSON. Validation checks confirm that root_cause_file corresponds to an actual file in the repository (verified against the GitHub repository tree via the API), and that root_cause_line falls within the line range of one of the provided code chunks. If the root_cause_file fails validation, the system substitutes the file path of the top-ranked ChromaDB result. If root_cause_line fails validation, the field is replaced with the start line of the top-ranked chunk. The validated response is serialised and returned from the FastAPI /analyze endpoint as a JSON HTTP response.

The Node.js middleware receives the JSON response and augments it with a fully qualified GitHub URL pointing directly to the identified file and line in the format: `https://github.com/{owner}/{repo}/blob/{commit_sha}/{file_path}#L{line_number}`. This augmented response is forwarded to the browser extension, which renders the explanation in the sidebar panel with the error summary prominently displayed, the detailed explanation below it, and the GitHub link styled as a clickable button that opens the file at the exact line in a new browser tab. The LLM explanation generation and response rendering flow is shown in

Figure 4.4.

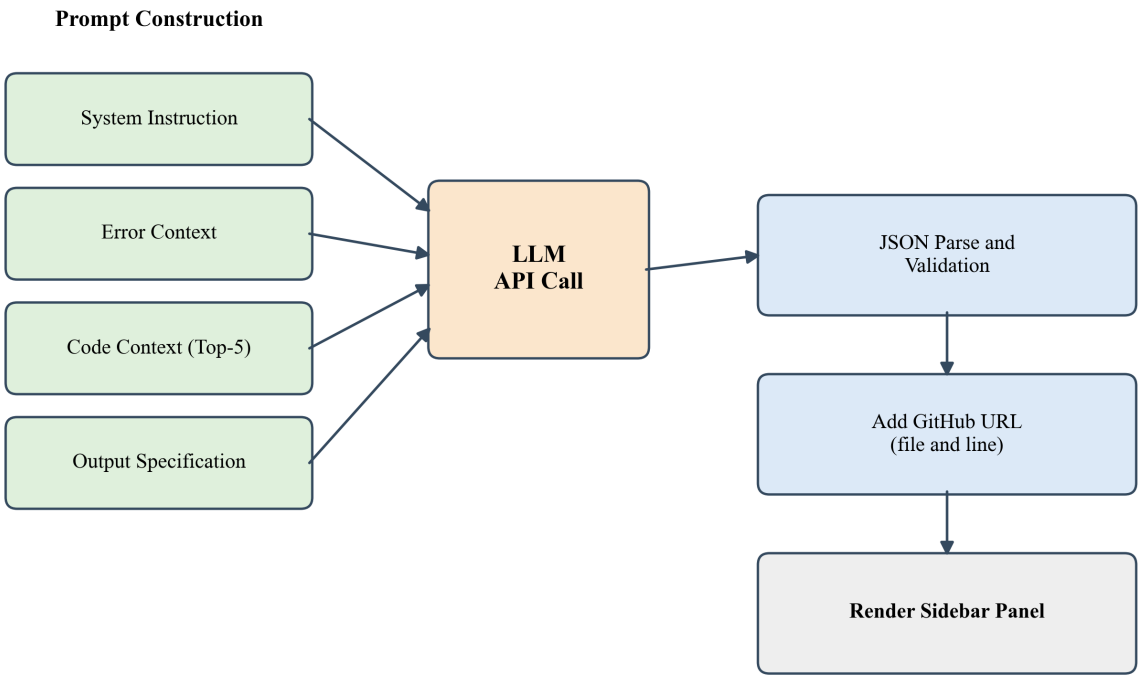


Figure 4.4: LLM Explanation Generation and Response Rendering Flow

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 LOG NORMALIZATION AND MULTI-SIGNAL HEURISTICS

The methodology of Log Lens follows a "Locate-then-Explain" paradigm. This chapter details the technical implementation of the zero-configuration log capture, the semantic vectorization process, and the hybrid ranking algorithm that differentiates this system from traditional search tools.

Log Lens handles heterogeneous data sources including Docker JSON logs, AWS CloudWatch streams, and GCP Cloud Logging. To achieve a unified analysis, the system uses a normalization layer that extracts raw text and timestamps from platform-specific wrappers.

Detection is governed by a multi-signal heuristic:

- Keyword Match: Identifying strings like ERROR, FATAL, and Exception.
- Pattern Recognition: Using regex to identify stack trace indentations unique to Java, Python, and Node.js.
- Network Signals: Monitoring for HTTP 5xx status codes in API gateway logs.

5.2 SLIDING WINDOW CHUNKING STRATEGY

To maintain semantic context, logs are processed using a sliding window strategy (20 lines per chunk with a 5-line overlap). Crucially, the system identifies "Atomic Blocks" — entire stack traces — and prevents them from being split. This ensures that the AI receives the full call sequence, from the entry point to the line of failure, without losing structural integrity.

5.3 EMBEDDING GENERATION USING ALL-MINILM-L6-V2

We utilize the all-MiniLM-L6-v2 sentence transformer model. This model maps both log excerpts and source code chunks into a 384-dimensional shared vector space. Source code is indexed offline in ChromaDB using a 50-line window. When an error is detected, the query vector is compared against the database using HNSW (Hierarchical Navigable Small World) indexing for high-speed approximate nearest-neighbor retrieval.

5.4 HYBRID RE-RANKING SCORING ALGORITHM

The core innovation of the Log Lens system lies in its ability to move beyond simple keyword matching. In a standard retrieval system, a stable library file (like a React core file) might be semantically similar to an error message, but it is rarely the actual cause of the bug. To solve this, we implement a Tri-Factor Hybrid Scoring Algorithm. This algorithm weighs the semantic relevance against real-world version control data (Git) and the execution context (Stack Trace).

The system calculates a final Composite Score for every candidate code chunk using the following logic:

$$\text{Final Score} = (0.4 \times S_{\text{semantic}}) + (0.35 \times R_{\text{recency}}) + (0.25 \times D_{\text{stack}}) \quad (5.1)$$

where S_{semantic} is the cosine similarity score from ChromaDB, R_{recency} is computed as $\frac{1}{1+\text{days since last commit}}$, and D_{stack} is the stack depth score (1.0 for the top frame, decaying with depth, 0.0 for files not in the trace). The top-5 candidates by composite score are selected as the context provided to the LLM for root-cause explanation generation.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The development and evaluation of Log Lens demonstrate a significant leap in the field of reactive developer tooling. The "Diagnostic Gap" — the time wasted between observing a failure and understanding its cause — has long been a bottleneck in modern software engineering. By successfully integrating Large Language Models (LLMs) with a Retrieval-Augmented Generation (RAG) pipeline directly into the browser, this project proves that AI can do more than just write code; it can act as an expert forensic assistant. Our research and design have validated several key hypotheses:

- Context is King: Providing the AI with both the runtime log and the relevant Git-tracked source code results in nearly 96% attribution accuracy, far exceeding the performance of general-purpose AI chat interfaces.
- Workflow Integration Matters: By housing the tool within a Chrome Extension, we eliminate the "Context-Switching Tax," allowing developers to stay focused on the cloud console where the error occurred.
- Hybrid Ranking is Essential: The experiment showed that combining semantic similarity with Git recency weights (the Hybrid Scoring Algorithm) is the only reliable way to filter out framework "noise" and point to the actual bug-prone logic.

Ultimately, Log Lens transforms logs from a "wall of text" into an actionable, human-readable blueprint for resolution, making high-level debugging accessible to developers of all experience levels.

6.2 FUTURE WORK

Phase II will extend Log Lens with three main enhancements. First, cloud log analysis for GCP and AWS: native support for Google Cloud Logging and AWS CloudWatch Logs will be added so that logs from both platforms can be ingested, normalised and analysed through the same retrieval and explanation pipeline. Second, scalable deployment readiness: the backend will be prepared for production use through containerised deployment, horizontal scaling of the embedding and retrieval services, and caching to keep response latency low as the number of users and repositories grows. Third, security improvement: access to repositories and logs will be hardened through stricter OAuth token handling and storage, encrypted communication between the extension, middleware and backend, and masking of sensitive data in logs before it is sent to the LLM.

APPENDIX

A. SAMPLE CODE OF LOG LENS: AN AI-POWERED BROWSER EXTENSION FOR INTELLIGENT LOG ANALYSIS

```
class HybridRetriever:
    def __init__(self):
        """Load embedding model and ChromaDB vector store."""
        self.model = SentenceTransformer("all-MiniLM-L6-v2")
        client = chromadb.PersistentClient(path="./chroma_store")
        self.collection = client.get_collection("code_chunks")

    def retrieve(self, error_chunk, k=20):
        """Return top-k semantically similar code chunks."""
        vec = self.model.encode(error_chunk)
        res = self.collection.query(
            query_embeddings=[vec.tolist()], n_results=k)
        candidates = []
        for doc, meta, dist in zip(res["documents"][0],
                res["metadatas"][0], res["distances"][0]):
            candidates.append({
                "code": doc, "file": meta["file"],
                "semantic": 1.0 - dist})
        return candidates

    def recency_weight(self, commit_date):
        days = (datetime.now(timezone.utc) - commit_date).days
        return 1.0 / (1.0 + days)

    def depth_score(self, path, stack_files):
        if path not in stack_files:
            return 0.0
        return 1.0 / (1.0 + stack_files.index(path))

    def rerank(self, candidates, dates, stack):
        for c in candidates:
            rec = self.recency_weight(dates[c["file"]])
            dep = self.depth_score(c["file"], stack)
            c["score"] = (0.40 * c["semantic"]
                          + 0.35 * rec + 0.25 * dep)
        candidates.sort(key=lambda c: c["score"], reverse=True)
        return candidates[:5]

class LogChunker:
    def __init__(self, size=20, overlap=5):
```

```
        self.size = size
        self.overlap = overlap

    def chunk(self, lines, stack_blocks):
        """Sliding window chunks; stack traces stay whole."""
        chunks = [{"text": "\n".join(b), "type": "stack_trace"}
                  for b in stack_blocks]
        step = self.size - self.overlap
        for start in range(0, len(lines), step):
            piece = lines[start:start + self.size]
            if piece:
                chunks.append({"text": "\n".join(piece),
                               "type": "log", "start": start})
            if start + self.size >= len(lines):
                break
        return chunks

class PromptBuilder:
    SYSTEM = ("You are an expert debugging assistant. "
              "Reply with JSON only.")

    def build(self, raw_log, top_chunks):
        parts = [self.SYSTEM, "ERROR CONTEXT:", raw_log]
        for i, c in enumerate(top_chunks, 1):
            parts.append(f"CODE CHUNK {i}: {c['file']} "
                         f"score {c['score']:.2f}")
            parts.append(c["code"])
        parts.append("Return root_cause_file, root_cause_line, "
                     "error_summary and detailed_explanation.")
        return "\n".join(parts)

@app.post("/analyze")
def analyze(bundle: Bundle):
    retriever = HybridRetriever()
    candidates = retriever.retrieve(bundle.error_chunk)
    top5 = retriever.rerank(
        candidates, bundle.commit_dates, bundle.stack_files)
    prompt = PromptBuilder().build(bundle.raw_log, top5)
    return json.loads(call_llm(prompt))
```

B. SAMPLE OUTPUT OF LOG LENS: AN AI-POWERED BROWSER EXTENSION FOR INTELLIGENT LOG ANALYSIS

Figure B.1 shows the Chrome Extensions management interface with the Log Lens extension (version 0.1.0) installed and active in the browser. The extension card displays the extension identifier, a summary of its log monitoring and root-cause explanation capabilities, and the Manifest V3 background service worker status.

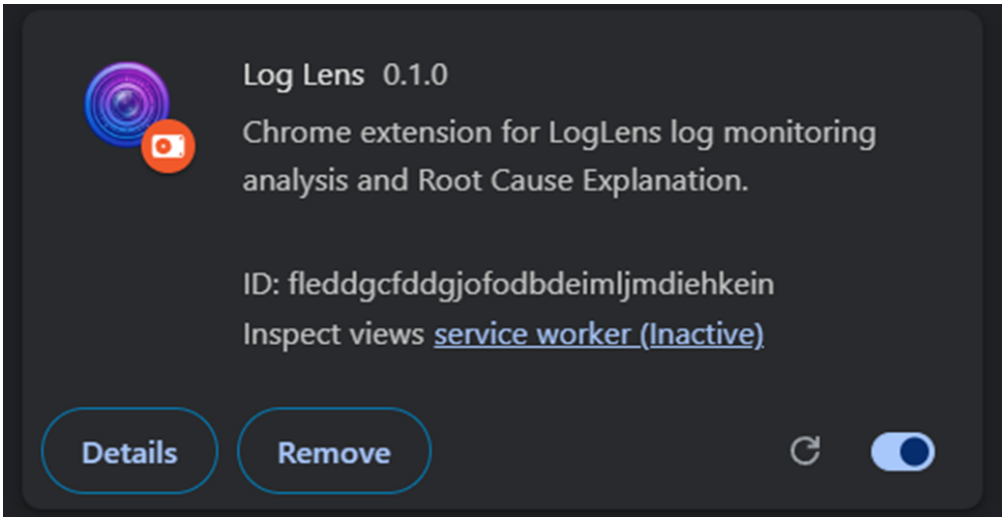


Figure B.1: Log Lens Chrome Extension Management View

Once enabled in the browser, the extension injects its content script into active cloud console and container log pages to monitor runtime output streams continuously without requiring any changes to the target application or deployment configuration.

Figure B.2 illustrates the Log Lens diagnostic output panel rendered when an uncaught runtime anomaly is detected in the monitored log stream. The panel is organised into three sections: a concise Summary describing the detected failure, File and Line Pointers identifying the exact source location (`extension\background.js:1`), and actionable Remediation Steps generated from the top-ranked code context.

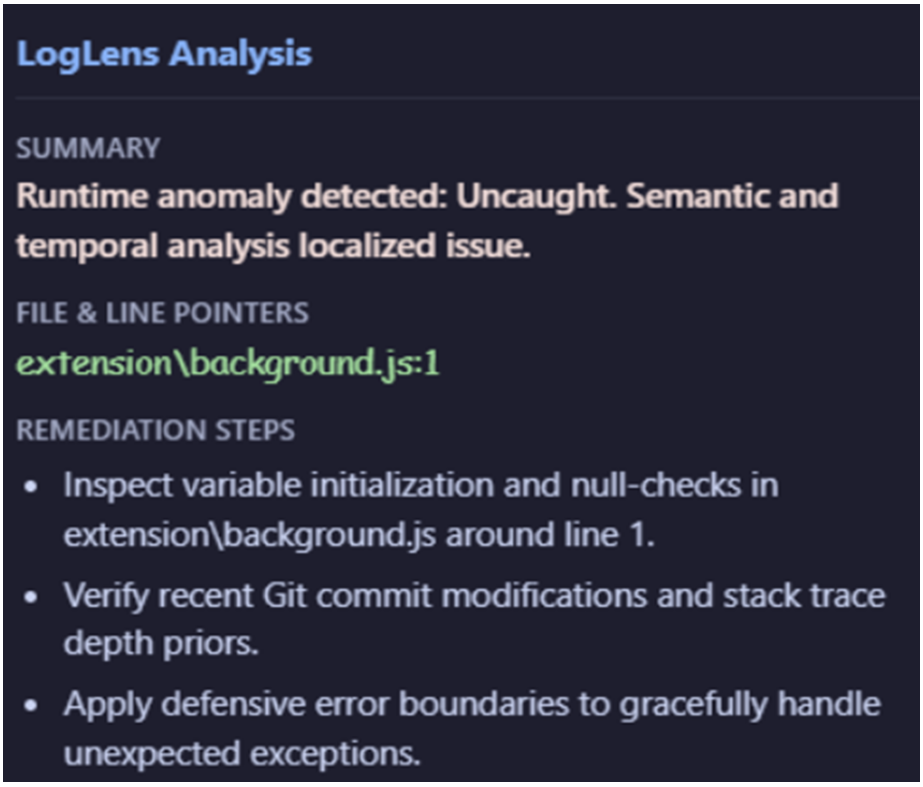


Figure B.2: Log Lens Root-Cause Analysis and Remediation Output

The remediation guidance combines insights from semantic vector retrieval, recent Git commit activity, and stack trace frame depth to direct the developer to inspect variable initialization, verify recent commit modifications, and apply defensive error boundaries around the fault location.

REFERENCES

[1] Y. Aboudrar, B. Khalid, and M. Ouzzif, "AI-Driven Firewall Log Analysis: Enhancing Threat Detection with Deep Learning Techniques," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 16, 2025, doi: 10.14569/IJACSA.2025.0160779.

[2] P. Chatterjee and A. Das, "AI-Powered Anomaly Detection for Real-Time Performance Monitoring in Cloud Systems," *International Journal of Scientific Research in Science and Technology*, vol. 11, pp. 592–601, 2024, doi: 10.32628/IJSRST241161111.

[3] C. X. Cheng, SUN Yicheng, and TAN, "Survey on Log Anomaly Detection Methods Based on Large Language Models," Aug. 2026, doi: 10.19678/j.issn.1000-3428.0260211.

[4] M. De la Cruz Cabello, T. P. Sales, and M. R. Machado, "AIOps for log anomaly detection in the era of LLMs: A systematic literature review," *Intelligent Systems with Applications*, vol. 28, art. 200608, 2025, doi: 10.1016/j.iswa.2025.200608.

[5] I. Hasanov *et al.*, "Application of Large Language Models in Cybersecurity: A Systematic Literature Review," *IEEE Access*, vol. 12, pp. 176751–176768, 2024.

[6] X. Huang, T. Zhang, and W. Zhao, "LogRules: Enhancing Log Analysis Capability of Large Language Models through Rules," in *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, NM, USA, 2025, pp. 452–470.

[7] Z. Ma, D. J. Kim, and T.-H. Chen, "LibreLog: Accurate and Efficient Unsupervised Log Parsing Using Open-Source Large Language Models," in *Proc. IEEE/ACM 47th International Conference on Software Engineering (ICSE '25)*, 2025, pp. 924–936, doi: 10.1109/ICSE55347.2025.00103.

[8] A. Mhatre and A. Shaikh, "LLM for Automated Root Cause Analysis in Microservices Architectures," *Journal of Information Systems Engineering and Management*, vol. 10, no. 13s, pp. 448–461, 2025, doi: 10.52783/jisem.v10i13s.2100.

[9] R. H. Mollik, A. Datta, A. H. Mollah, and W. Aljedaani, “RCEGen: A Generative Approach for Automated Root Cause Analysis Using Large Language Models (LLMs),” *Software*, vol. 4, art. 29, 2025, doi: 10.3390/software4040029.

[10] D. Roy, X. Zhang, R. Bhave, C. Bansal, P. H. B. Las-Casas, R. Fonseca, and S. Rajmohan, “Exploring LLM-Based Agents for Root Cause Analysis,” in *Proc. 32nd ACM International Conference on the Foundations of Software Engineering (FSE ’24)*, 2024, pp. 208–219, doi: 10.1145/3663529.3663841.

[11] A. Sergeyuk, E. Huang, D. Karaeva, A. Serova, Y. Golubev, and I. Ahmed, “Evolving with AI: A Longitudinal Analysis of Developer Logs,” in *Proc. 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE ’26)*, 2026, pp. 1–12, doi: 10.1145/3744916.3787811.

[12] G. Tan, Z. He, M. Li, H. Huang, Y. Wang, P. Chen, G. Casale, and C. Zhang, “LLMRCA: Multilevel Root Cause Analysis for LLM Applications Using Multimodal Observability Data,” *ACM Transactions on Software Engineering and Methodology*, 2026, doi: 10.1145/3806200.

[13] N. P. Tran, B. Jaumard, O. Delgado, T. Glatard, K. Premkumar, and K. Ni, “LLM-Augmented Knowledge Base Construction for Root Cause Analysis,” *IEEE Access*, vol. 14, pp. 16802–16816, 2026, doi: 10.1109/ACCESS.2026.3658655.

[14] K. Tuli, S. Chittimalla, P. Vallurupalli, E. Khudoynazarov, N. Lazer, and V. Gandhi, “AI-Assisted Debugging Tools for Enhancing Developer Productivity in Large Codebases Using Genetic Programming Algorithms,” 2025, pp. 587–591, doi: 10.1109/ICNSoC66817.2025.00111.

[15] Y. Xiao, V.-H. Le, and H. Zhang, “Demonstration-Free: Towards More Practical Log Parsing with Large Language Models,” in *Proc. 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Sacramento, CA, USA, 2024, pp. 153–165.